CONTACT MANAGER – FINAL PROJECT DOCUMENTATION

Team Information

Team Leader: SANTHOSH C

Team ID: SWTID-2026-8138

Team Members

THARUN M P

ROHINTH M

VIJAY AROKIYA RAJ T

Project Documentation

ConnectHub

PROJECT OVERVIEW

The ConnectHub is a backend web application designed to solve the common problem of scattered contact information across multiple platforms. It provides users with a centralized, organized system to store, search, and categorize their professional and personal contacts.

Key Objectives:

- Provide secure user authentication and data isolation
- Enable efficient contact management through CRUD operations
- Implement flexible categorization using tags
- Offer fast search and filtering capabilities
- Deliver a responsive, cross-device user experience

Target Users:

- Freelancers and consultants managing client relationships
- Small business owners tracking vendor and customer contacts
- Students and young professionals building their network

- Event organizers managing attendee and volunteer information
- Anyone seeking a simple, privacy-focused contact management solution

PROJECT INSTRUCTION

1 Install Node.js (If Not Installed)

Download and install Node.js from: https://nodejs.org/

After installation, verify:

```
node -v
npm -v
```

2 Create a Backend Project Folder

• Choose or create a directory where you want your backend.

• Open your terminal or command prompt.

• Navigate to the selected directory:

```
cd your-project-folder
```

• Create a new backend folder:

```
mkdir server
cd server
```

3 Initialize Node Project

Run: npm init -y

This will create a package.json file.

4 Install Required Dependencies

Install Express:

```
npm install express
npm install mongoose
```

For development (auto-restart server):

```
npm install nodemon --save-dev
```

5 Create Basic Server File

Inside the server folder:

• Create a file called: server.js

Add this code :

6⃣ Enable ES Modules (Important)

Open package.json and add: "type": "module"

Example:

7⃣ Start the Backend Server

Run: npm run dev

OR if not using nodemon: node server.js

8⃣ Open in Browser

Open: http://localhost:5000

You should see: Backend is running

PRE-REQUISITES

Software Requirements

Knowledge Requirements

- JavaScript ES6+ syntax and concepts
- Node.js and Express.js for RESTful API development
- MongoDB and Mongoose ODM
- JWT authentication and bcrypt password hashing
- Git version control workflow

PROJECT THUMBNAIL

PROJECT ARCHITECTURE

High-level architecture diagram illustrating the three-tier backend structure.

Node.js + Express Backend

• RESTful API architecture

• JWT authentication middleware

• MVC pattern (Models, Controllers, Routes)

• Input validation and error handling

MongoDB Database

• Document-based NoSQL storage

• Mongoose ODM for schema modeling

• Two collections: Users and Contacts

• Indexed fields for optimized queries

TECHNICAL ARCHITECTURE

Backend Architecture (Node.js + Express)

- Controllers: Business logic for authentication and CRUD operations
- Models: Mongoose schemas defining data structure
- Routes: API endpoint definitions with HTTP methods
- Middleware: JWT verification, error handling, validation
- Configuration: Database connection and environment variables

Database Architecture (MongoDB)

- Users Collection: _id, name, email, password (hashed), timestamps
- Contacts Collection: _id, userId (FK), name, email, phone, notes, tags[], timestamps
- Relationships: Many-to-One (Contacts → User)
- Indexes: email (unique), userId, name, tags (for search)

ER Diagram

Features

Core Features

Roles and Responsibility

User -

Registration - Register on the application using the name, email , password

Login - Login on the application using the email and registered password

Search - Search for a contact using name , email or tags

Add Contact - Add new contact - name , email , phone , notes , tags

Edit Contact - Edit the existing contact

Delete Contact - Delete any contact

User Flow

Project Setup and Configuration

Folder Structure

Project Setup

1. Install required tools and software:

 Node.js.

 MongoDB.

2. Create project folders and files:

 Server folders.

3. Install Packages:

Backend npm Packages

 Express.

 Mongoose.

 Cors.

Backend Development

 Setup express server

 Create index.js file in the server (backend folder).

 Create a .env file and define port number to access it globally.

 Configure the server by adding cors, body-parser.

 User Authentication:

• Create routes and middleware for user registration, login, and

• Set up authentication middleware to protect routes that require user authentication.

 Define API Routes:

• Create separate route files for different API functionalities such as user authentication and crud operation for contacts

• Implement route handlers using Express.js to handle requests and interact with the database.

 Implement Data Models:

• Define Mongoose schemas for the different data entities like user and contact

• Create corresponding Mongoose models to interact with the MongoDB database.

• Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

 Error Handling:

• Implement error handling middleware to catch and handle any errors that occur during the API requests.

• Return appropriate error responses with relevant error messages and HTTP status codes.

Database Configuration and Setup

1. Configure MongoDB:

 Install Mongoose.

 Create database connection.

 Create Schemas & Models.

2. Connect database to backend:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

3. Configure Schema:

Firstly, configure the Schemas for MongoDB database, to store the data in such pattern. Use the data from the ER diagrams to create the schemas.

The schemas are looks like for the Application.

Project Execution

Execution Steps

Open the project folder in the terminal

Make sure you have a start script in the package.js file

Use the command npm start to start the server

You will the Running on PORT message on the successfully starting the server

You will also see the database connected message if the database is connected successfully

Use the Thunder client or Postman for using the created api's

Output Screenshots

Register Api

Login Api

Create New Contact

Update Contact

Delete Contact

Search Contact

Get The Current User

Project Demo

Contact Manager.webm

Code Link

Contact-Manager

GitHub Repository Link

https://github.com/bsccsaisanthosh-cmyk/contact-manager.git

Project Demo Link

https://drive.google.com/file/d/1eHSEcL8bqhv090S9xLlcQscCaSxjf9XY/view?usp=drivesdk